Потоци

Трифон Трифонов

Обектно-ориентирано програмиране,
спец. Компютърни науки, 1 поток, 2021/22 г.

11–13 април 2022 г.

Тази презентация е достъпна под лиценза Creative Commons Признание-Некомерсиално-Споделяне на споделеното 4.0 Международен



"Blue Stream" от Fitz Gerard Villafuerte (https://flic.kr/p/7FmKBb), CC BY-NC-ND 2.0

Взаимодействие на две програми

Програма A пресмята поредица от данни

- простите числа
- кадри от видео клип
- списък от постове във Facebook/Twitter

Взаимодействие на две програми

Програма А пресмята поредица от данни

- простите числа
- кадри от видео клип
- списък от постове във Facebook/Twitter

Програма Б обработва поредица от данни

- търси числа-близнаци
- прави снимки на “интересни” моменти от клипа
- събира всички постове с линк към YouTube

Взаимодействие на две програми

Програма А пресмята поредица от данни

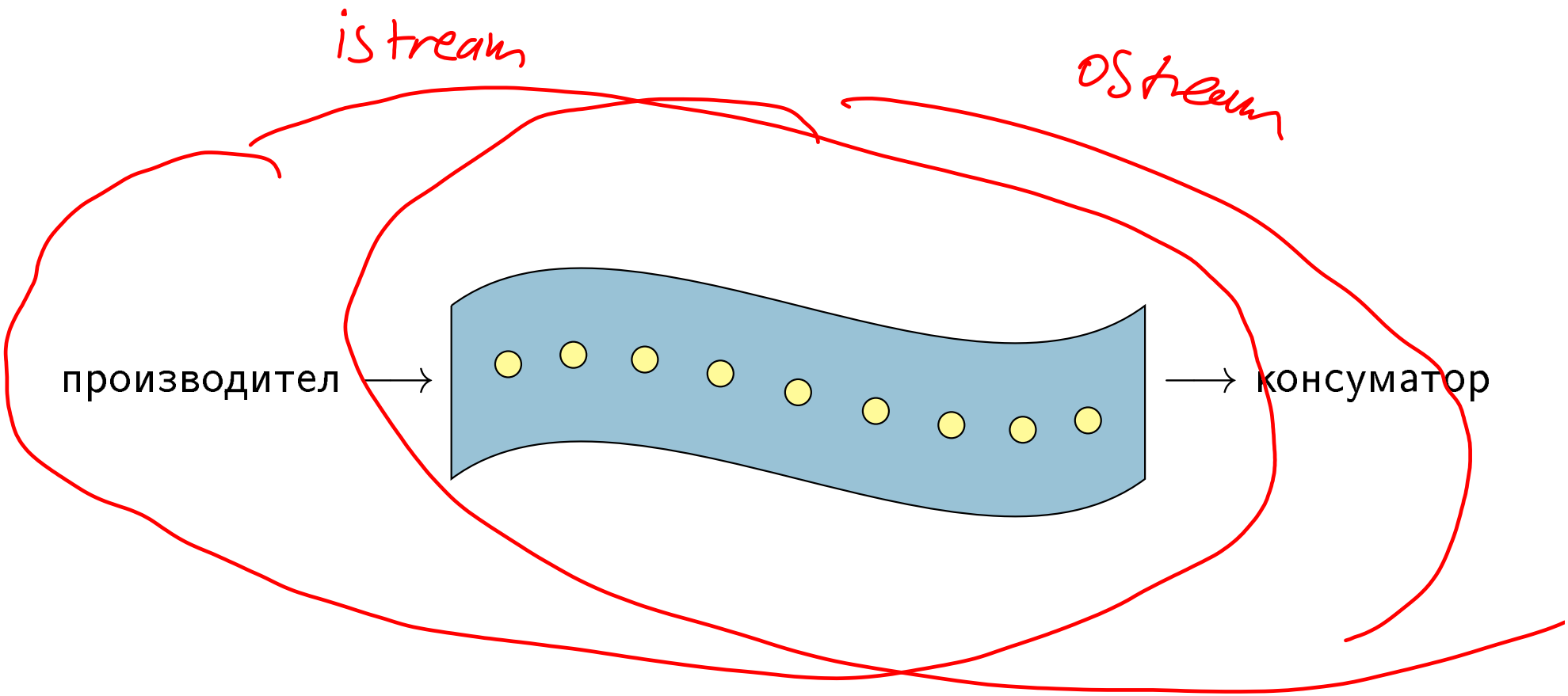
- простите числа
- кадри от видео клип
- списък от постове във Facebook/Twitter

Програма Б обработва поредица от данни

- търси числа-близнаци
- прави снимки на “интересни” моменти от клипа
- събира всички постове с линк към YouTube

Как да организираме работата на двете програми?

Абстракцията поток



Обектно-ориентиран подход

```
std::cin >> number >> character >> string;
```

```
file << student << list << tree;
```

```
while (stream1 >> x) stream2 << f(x);
```

Конвейерна обработка

- събирането на няколко потока в един голям поток

Конвейерна обработка

- събирането на няколко потока в един голям поток
- ефективна паралелна обработка

Конвейерна обработка

- събирането на няколко потока в един голям поток
- ефективна паралелна обработка
- саморегулиращ се механизъм

Конвейерна обработка

- събирането на няколко потока в един голям поток
- ефективна паралелна обработка
- саморегулиращ се механизъм
- **Пример:** Unix pipes

Конвейерна обработка

- събирането на няколко потока в един голям поток
- ефективна паралелна обработка
- саморегулиращ се механизъм
- **Пример:** Unix pipes
- `ls | grep new | wc -l`

Конвейерна обработка

- събирането на няколко потока в един голям поток
- ефективна паралелна обработка
- саморегулиращ се механизъм
- **Пример:** Unix pipes
- `ls | grep new | wc -l`
- Файловете като производители или консуматори на данни

Поточен буфер

- Какво представлява буферът?

Поточен буфер

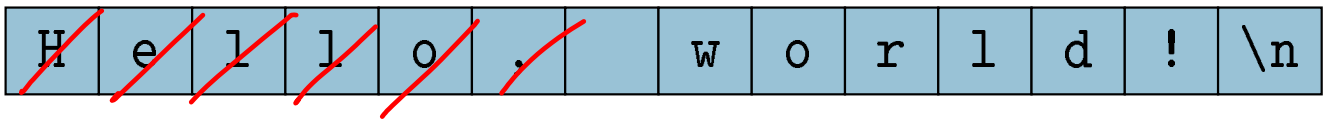
- Какво представлява буферът?
- Кога е нужен буфер?

Поточен буфер

- Какво представлява буферът?
- Кога е нужен буфер?
- Кога буферът вреди?

Поточен буфер

- Какво представлява буферът?
- Кога е нужен буфер?
- Кога буферът вреди?



std::cin >> s;

Стандартни потоци и пренасочване

- Стандартен изходен поток `std::cout` (`stdout`)
  - Пренасочване на изхода:
  - `ls > filelist.txt`

Стандартни потоци и пренасочване

- Стандартен изходен поток `std::cout` (`stdout`)
  - Пренасочване на изхода:
  - `ls > filelist.txt`
- Стандартен входен поток `std::cin` (`stdin`)
  - Пренасочване на вход и на изход:
  - `grep password < email.txt > password.txt`

Стандартни потоци и пренасочване

- Стандартен изходен поток `std::cout` (`stdout`)
  - Пренасочване на изхода:
  - `ls > filelist.txt`
- Стандартен входен поток `std::cin` (`stdin`)
  - Пренасочване на вход и на изход:
  - `grep password < email.txt > password.txt`
- Стандартен поток за грешки `std::cerr` (`stderr`)
  - Пренасочване на изход за грешки:
  - `mv *.dat /data 2> errors.txt`

Стандартни потоци и пренасочване

- Стандартен изходен поток `std::cout` (`stdout`)
  - Пренасочване на изхода:
  - `ls > filelist.txt`
- Стандартен входен поток `std::cin` (`stdin`)
  - Пренасочване на вход и на изход:
  - `grep password < email.txt > password.txt`
- Стандартен поток за грешки `std::cerr` (`stderr`)
  - Пренасочване на изход за грешки:
  - `mv *.dat /data 2> errors.txt`
- Стандартен поток за дневник `std::clog` (отново `stderr`)

Форматиран и неформатиран вход/изход

- Текстова и двоична информация

Форматиран и неформатиран вход/изход

- Текстова и двоична информация
- ASCII (`char`)

Форматиран и неформатиран вход/изход



- Текстова и двоична информация
- ASCII (`char`)
- Служебни символи

10 LF

13 CR

13 10 CRLF

Форматиран и неформатиран вход/изход

- Текстова и двоична информация
- ASCII (`char`)
- Служебни символи
- Кодиращи таблици

CP1251

Форматиран и неформатиран вход/изход

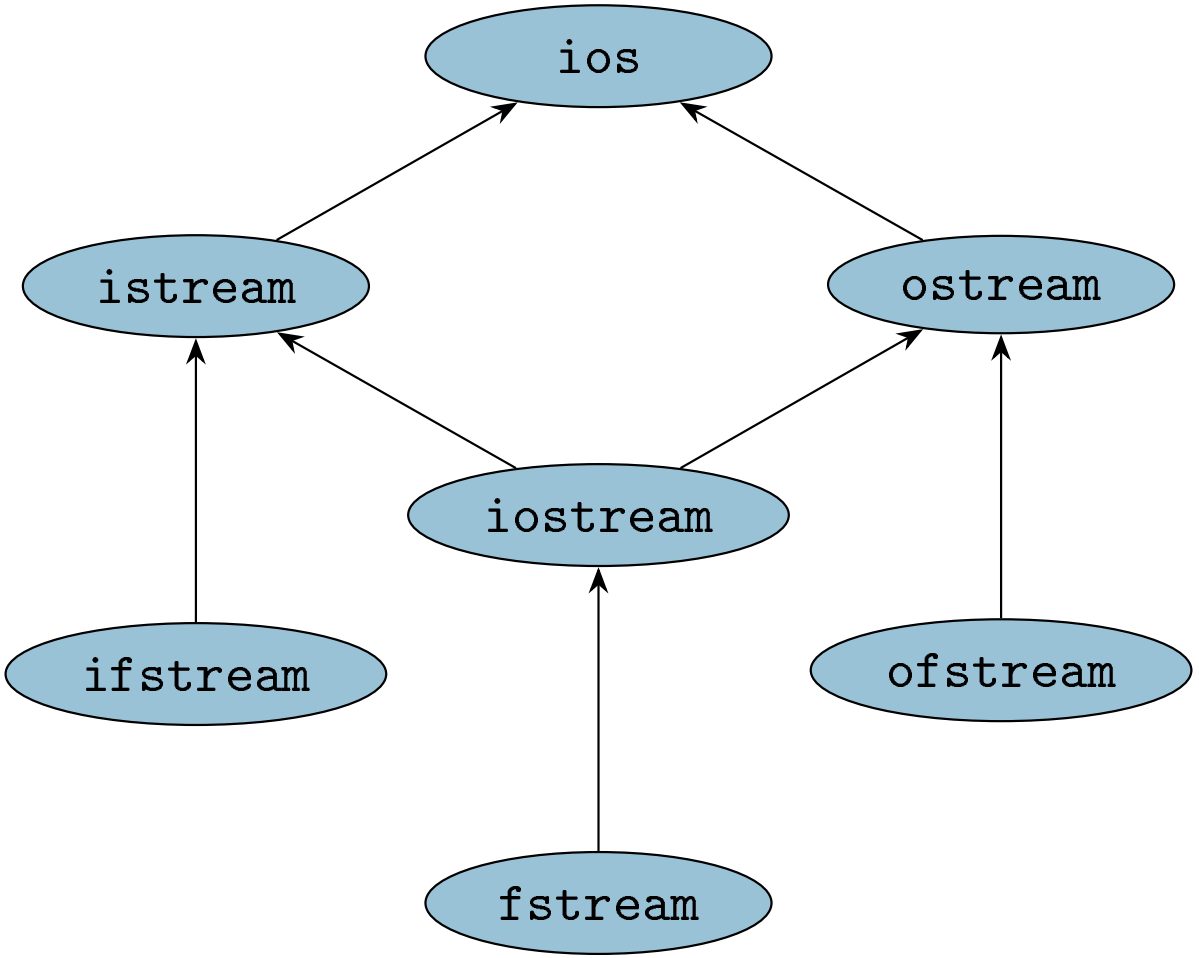
- Текстова и двоична информация
- ASCII (`char`)
- Служебни символи
- Кодиращи таблици
- Unicode (`wchar_t`)

25 65536

Форматиран и неформатиран вход/изход

- Текстова и двоична информация
- ASCII (`char`)
- Служебни символи
- Кодиращи таблици
- Unicode (`wchar_t`)
- UTF-8

Поточна йерархия в C++



Изход на поток

Неформатиран изход:

```
std::ostream& put(char);
std::ostream& write(const char*, std::streamsize);
```

Изход на поток

std::cout.put(15).put(18).put(65);

Неформатиран изход:

byte

```
std::ostream& put(char);
std::ostream& write(const char*, std::streamsize);
```

byte

Форматиран изход:

```
std::ostream& operator<<(std::ostream&, T);
```

Вход от поток

Неформатиран вход:

std::cin.get(c).get(c);

```
int get();
std::istream& get(char&);
std::istream& get(char*, std::streamsize, char = '\n');
std::istream& getline(char*, std::streamsize, char = '\n');
std::streamsize gcount() const;
std::istream& read(char*, std::streamsize);
```

Вход от поток

Неформатиран вход:

```
int get();
std::istream& get(char&);
std::istream& get(char*, std::streamsize, char = '\n');
std::istream& getline(char*, std::streamsize, char = '\n');
std::streamsize gcount() const;
std::istream& read(char*, std::streamsize);
```

Форматиран вход:

```
std::istream& operator>>(std::istream&, T&);
```

Вход от поток

Неформатиран вход:

```
int get();
std::istream& get(char&);
std::istream& get(char*, std::streamsize, char = '\n');
std::istream& getline(char*, std::streamsize, char = '\n');
std::streamsize gcount() const;
std::istream& read(char*, std::streamsize);
```

Форматиран вход:

```
std::istream& operator>>(std::istream&, T&);
```

Допълнителни функции:

```
int peek();
std::istream& putback(char);
```

Низови потоци

```
#include <sstream>
```

Входен поток от низ: `std::istringstream`

Пример:

```
char s[] = "1 2 3";
std::istringstream iss(s);
int a, b, c;
iss >> a >> b >> c;
```

Низови потоци

```
#include <sstream>
```

Входен поток от низ: `std::istringstream`

Пример:

```
char s[] = "1 2 3";
std::istringstream iss(s);
int a, b, c;
iss >> a >> b >> c;
```

Изходен поток към низ: `std::ostringstream`

Пример:

```
std::ostringstream oss;
oss << 1.2 << ' ' << 3.4;
std::cout << oss.str();
```

Състояние на поток

Флагове за състояние:

std::ios::iostate	goodbit	badbit	eofbit	failbit
	0	1	2	4

Състояние на поток

Флагове за състояние:

std::ios::iostate	goodbit	badbit	eofbit	failbit
	0	1	2	4

```
bool good() const; bool eof() const;
bool fail() const; bool bad() const;
std::ios::iostate rdstate() const;

void clear(std::ios::iostate = 0);
```

Примери:

```
if (std::cin.rdstate() &
    (std::ios::eofbit | std::ios::badbit)) ...
std::cin.clear(std::ios::failbit);
if(std::cin)...          if(!std::cin)...
```

Потокови манипулатори

```
#include <iomanip>
stream << data1 << manipulator << data2;
```

- Манипулатори за изход: `endl`, `ends`, `flush`

Потокови манипулатори

```
#include <iomanip>
stream << data1 << manipulator << data2;
```

- Манипулатори за изход: `endl`, `ends`, `flush`
- Манипулатори за бройна система: `hex`, `oct`, `dec`

Потокови манипулатори

```
#include <iomanip>
stream << data1 << manipulator << data2;
```

- Манипулатори за изход: `endl`, `ends`, `flush`
- Манипулатори за бройна система: `hex`, `oct`, `dec`
- Манипулатори за поле: `setw`, `setfill`, `left`, `right`, `internal`

Потокови манипулатори

```
#include <iomanip>
stream << data1 << manipulator << data2;
```

- Манипулатори за изход: `endl`, `ends`, `flush`
- Манипулатори за бройна система: `hex`, `oct`, `dec`
- Манипулатори за поле: `setw`, `setfill`, `left`, `right`, `internal`
- Манипулатори за дробни числа: `fixed`, `scientific`, `setprecision`

Потокови манипулатори

```
#include <iomanip>
stream << data1 << manipulator << data2;
```

- Манипулатори за изход: `endl`, `ends`, `flush`
- Манипулатори за бройна система: `hex`, `oct`, `dec`
- Манипулатори за поле: `setw`, `setfill`, `left`, `right`, `internal`
- Манипулатори за дробни числа: `fixed`, `scientific`, `setprecision`
- Манипулатори за формат: `setiosflags`, `setbase`

Потокови манипулатори

```
#include <iomanip>
stream << data1 << manipulator << data2;
```

- Манипулатори за изход: `endl`, `ends`, `flush`
- Манипулатори за бройна система: `hex`, `oct`, `dec`
- Манипулатори за поле: `setw`, `setfill`, `left`, `right`, `internal`
- Манипулатори за дробни числа: `fixed`, `scientific`, `setprecision`
- Манипулатори за формат: `setiosflags`, `setbase`
- ...и много други